

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT
TP. HỒ CHÍ MINH
Khoa Điện - Điện tử
Bộ môn Điện tử - Thông tin

Thực hành Học máy và Trí tuệ nhân tạo

Bài 3: Deep Learning - Multilayer Perception

Giảng viên: TS. Huỳnh Thế Thiện & PGS. TS Trương Ngọc Sơn

Tháng 03, 2026

1 Mục tiêu

Sau khi hoàn thành bài thực hành này, sinh viên có thể:

- Hiểu kiến trúc của Multilayer Perceptron (MLP)
- Xây dựng mô hình neural network bằng PyTorch
- Huấn luyện mô hình bằng backpropagation
- Đánh giá mô hình classification
- Thử nghiệm các kỹ thuật cải thiện mô hình

2 Giới thiệu bài toán

Trong bài thực hành này chúng ta sẽ giải quyết bài toán:

Nhận dạng chữ số viết tay

Dataset sử dụng là:

MNIST Handwritten Digits

Dataset gồm:

- 70,000 ảnh chữ số viết tay
- kích thước mỗi ảnh: 28x28 pixels

- 10 lớp: các chữ số từ 0 đến 9

Mỗi ảnh được biểu diễn bằng:

$$28 \times 28 = 784$$

features.

Do đó ta có thể sử dụng ****MLP để phân loại****.

3 Chuẩn bị môi trường

Import các thư viện cần thiết.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
import torch.optim as optim

from torch.utils.data import DataLoader
from torchvision import datasets
from torchvision import transforms
```

4 Tải dataset

```
transform = transforms.ToTensor()

train_dataset = datasets.MNIST(
    root='./data',
    train=True,
    download=True,
    transform=transform
)

test_dataset = datasets.MNIST(
    root='./data',
    train=False,
```

```
download=True,
transform=transform
)
```

5 DataLoader

```
train_loader = DataLoader(
train_dataset,
batch_size=64,
shuffle=True
)
```

```
test_loader = DataLoader(
test_dataset,
batch_size=64,
shuffle=False
)
```

6 Hiển thị một số ảnh

```
images, labels = next(iter(train_loader))

plt.imshow(images[0].squeeze(), cmap='gray')
plt.title(labels[0].item())
plt.show()
```

7 Xây dựng mô hình MLP

```
class MLP(nn.Module):

    def __init__(self):

        super(MLP, self).__init__()

        self.fc1 = nn.Linear(784, 256)
        self.fc2 = nn.Linear(256, 128)
        self.fc3 = nn.Linear(128, 10)

        self.relu = nn.ReLU()
```

```

def forward(self, x):

    x = x.view(-1, 784)

    x = self.relu(self.fc1(x))
    x = self.relu(self.fc2(x))
    x = self.fc3(x)

    return x

```

Khởi tạo mô hình.

```
model = MLP()
```

8 Loss function và Optimizer

```
criterion = nn.CrossEntropyLoss()
```

```
optimizer = optim.Adam(
model.parameters(),
lr=0.001
)
```

9 Huấn luyện mô hình

```
epochs = 5
```

```

for epoch in range(epochs):

    for images, labels in train_loader:

        outputs = model(images)

        loss = criterion(outputs, labels)

        optimizer.zero_grad()

        loss.backward()

```

```
optimizer.step()

print("Epoch:", epoch, "Loss:", loss.item())
```

10 Đánh giá mô hình

```
correct = 0
total = 0

with torch.no_grad():

    for images, labels in test_loader:

        outputs = model(images)

        _, predicted = torch.max(outputs, 1)

        total += labels.size(0)

        correct += (predicted == labels).sum().item()

accuracy = 100 * correct / total

print("Test accuracy:", accuracy)
```

11 Hiển thị dự đoán

```
images, labels = next(iter(test_loader))

outputs = model(images)

_, predicted = torch.max(outputs, 1)

plt.imshow(images[0].squeeze(), cmap='gray')
plt.title("Predicted: {}".format(predicted[0].item()))
plt.show()
```

12 Bài tập

12.1 Bài tập 1: Data Exploration

Thực hiện các yêu cầu sau:

- In kích thước của train dataset
- In kích thước của test dataset
- Hiển thị 10 ảnh đầu tiên của dataset

12.2 Bài tập 2: Hiểu kiến trúc mạng

Vẽ sơ đồ kiến trúc của mạng neural network:

- Input layer
- Hidden layers
- Output layer

Xác định số lượng tham số của mỗi layer.

12.3 Bài tập 3: Thay đổi kiến trúc mạng

Thay đổi kiến trúc MLP:

- thêm 1 hidden layer
- thay đổi số neuron

Ví dụ:

- $784 \rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow 10$

So sánh kết quả với mô hình ban đầu.

12.4 Bài tập 4: Activation Functions

Thử các activation function khác nhau:

- ReLU
- Tanh
- LeakyReLU

So sánh accuracy.

12.5 Bài tập 5: Batch Size

Thử các giá trị batch size:

- 32
- 64
- 128

So sánh:

- tốc độ huấn luyện
- độ chính xác

12.6 Bài tập 6: Learning Rate

Thử các giá trị learning rate:

- 0.1
- 0.01
- 0.001
- 0.0001

Vẽ biểu đồ loss theo epoch.

12.7 Bài tập 7: Dropout

Thêm Dropout để tránh overfitting.

Ví dụ:

```
self.dropout = nn.Dropout(0.5)
```

So sánh kết quả trước và sau khi dùng Dropout.

12.8 Bài tập 8: Training Epochs

Huấn luyện mô hình với các giá trị:

- 5 epochs
- 10 epochs
- 20 epochs

Vẽ biểu đồ:

- Training loss

- Test accuracy

12.9 Bài tập 9: Confusion Matrix

Tính confusion matrix cho mô hình.

Sử dụng:

```
from sklearn.metrics import confusion_matrix
```

Vẽ heatmap của confusion matrix.

12.10 Bài tập 10: Cải thiện mô hình

Sinh viên cần thử các phương pháp sau:

- thay đổi kiến trúc mạng
- thêm dropout
- thay đổi optimizer
- điều chỉnh learning rate

Mục tiêu:

Đạt accuracy $> 97\%$ trên tập test